



AI Debate Partner

Full Technical Project Plan & Build Guide

Powered by an open-source Hugging Face LLM

React + Vite

Node.js + Express

Hugging Face Inference

PostgreSQL + Prisma

Web Speech API

JWT Auth

Resume / Portfolio Project • Prepared for Bhushan Uparikar

AI Debate Partner

Pick any topic → the AI takes the opposite side and debates you in real time, scores your arguments, flags logical fallacies, and gives improvement tips.

1. Project Overview

What it is: An educational web app where a user chooses a debate topic and a side (Pro/Con). The AI argues the *opposite* side, and after each of the user's arguments it acts as a judge — producing a numeric score, detecting logical fallacies, and giving concrete tips to improve.

Why it stands out on a resume: It is real AI engineering (a dual-role LLM system with a structured scoring rubric), full-stack ownership (frontend + API + database + deploy), and a genuinely useful product angle beyond a plain chatbot.

Three things this project proves to a recruiter:

1. **Real AI integration** — structured prompting, an LLM-as-judge scoring layer, JSON output parsing, and a provider-swappable architecture.
2. **Full-stack skill** — React frontend, Node/Express API, PostgreSQL persistence, authentication, deployment.
3. **Product thinking** — the argument-scoring + fallacy-detection feature is the differentiator. It is the headline talking point in interviews.

2. Technology Stack & Why

Layer	Technology	Why this choice
Frontend	React + Vite + TypeScript	Fast dev server, component model fits a chat UI, TypeScript signals engineering rigor to reviewers.
Styling	Tailwind CSS	Build a clean, responsive UI quickly without writing large CSS files.
Backend	Node.js + Express + TypeScript	Lightweight, familiar, huge ecosystem. Keeps the AI token/API key on the server, never in the browser.
AI Model	Hugging Face Inference (open LLM, e.g. Llama 3.3 / Qwen 2.5)	Free tier / open-source. Called only from the backend. Wrapped behind one swappable function so the provider can be changed without touching app logic.
Database	PostgreSQL + Prisma ORM	Relational data (users → debates → turns → scores) maps perfectly to SQL. Prisma gives type-safe queries + easy migrations.
Voice mode	Web Speech API	Built into the browser — speech-to-text and text-to-speech at zero cost, no external API.
Auth	JWT (jsonwebtoken + bcrypt)	Stateless, simple, industry-standard. Lets users save and reopen debate history.
Hosting	Vercel (frontend) · Render (backend) · Neon (Postgres)	All have free tiers → the whole project runs at \$0.

Key security rule: The Hugging Face token lives **only on the backend** in a `.env` file. The React app never sees it — it only talks to your own Express API. Exposing an API key in frontend code is an instant red flag to reviewers and a real security hole.

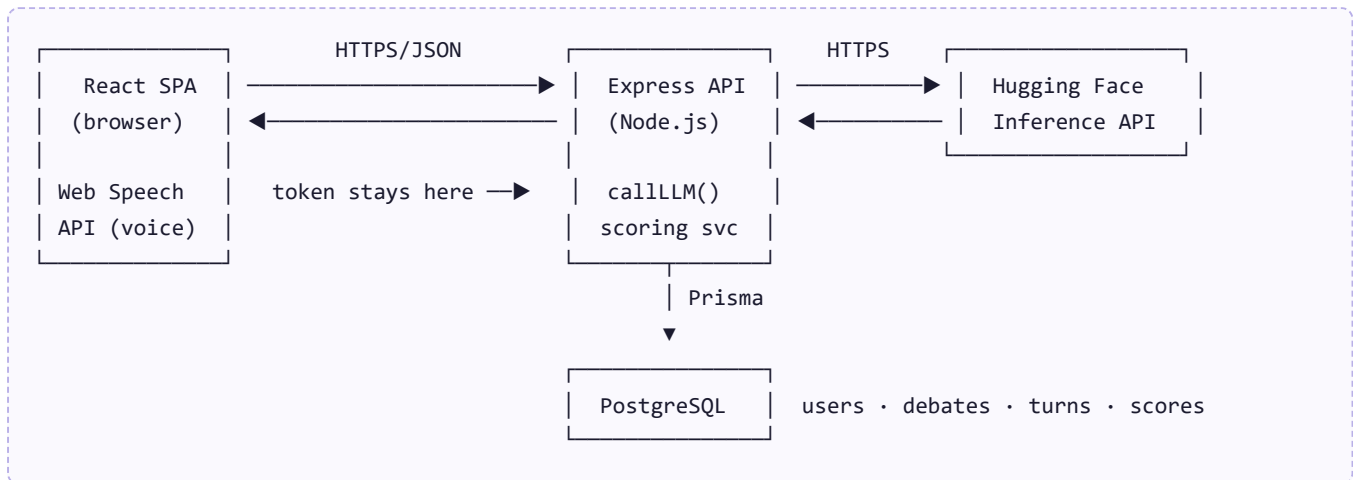
3. Choosing the Hugging Face Model

The app needs two things from the model: **good reasoning** (to argue and to judge fairly) and **reliable JSON output** (for the scoring). Bigger instruct-tuned models do both far better than tiny ones.

Role	Recommended model	Why
Primary (debate + scoring)	<code>meta-llama/Llama-3.3-70B-Instruct</code>	Strong reasoning, follows the "opposite side" persona and the scoring rubric well. Available via HF Inference Providers.
Cheaper / faster alt	<code>Qwen/Qwen2.5-7B-Instruct</code> or <code>mistralai/Mistral-7B-Instruct</code>	Fully free on the serverless tier, fast, fine for a demo. Weaker on nuanced scoring.
Local / offline (optional)	Run <code>Qwen2.5</code> via Ollama	100% free, no rate limits, no token — needs a decent CPU/GPU.

Reality check on "free": HF's serverless Inference API is rate-limited and small models don't always guarantee valid JSON. Mitigation is built into the design (Section 6): the scorer request asks for strict JSON, and the backend has a **JSON-repair + retry** wrapper so a malformed score never crashes the app.

4. System Architecture & Logic Flow



End-to-end flow of ONE debate turn

1. User types/speaks an argument in the React chat box.
2. Frontend POSTs { debateId, argument } to POST /api/debate/:id/turn (with JWT).
3. Express saves the user's Turn row in Postgres.
4. Express calls callLLM() TWICE, in parallel:
 - (a) DEBATER → generates the AI's opposite-side rebuttal.
 - (b) JUDGE → returns strict JSON: scores + fallacies + tips.
5. Backend parses/repairs the Judge JSON, saves an AI Turn + a Score row.
6. Backend responds: { aiRebuttal, score, fallacies, tips, runningTotal }.
7. Frontend renders the rebuttal in chat + a score card beside it.
8. (Voice mode) Web Speech API reads the AI rebuttal aloud.

Why two separate AI roles?

Mixing "argue against me" and "grade me fairly" in one prompt produces a biased judge. Splitting them into a **Debater** call and a **Judge** call keeps scoring objective and makes each prompt simpler and more reliable — this separation is itself a strong interview talking point.

5. Database Model (PostgreSQL / Prisma)

Table	Key fields	Purpose
User	id, email, passwordHash, createdAt	Account + auth.
Debate	id, userId, topic, userSide, aiSide, status, createdAt	One debate session.
Turn	id, debateId, speaker (user/ai), content, turnNumber, createdAt	Every message in the debate.
Score	id, turnId, logic, evidence, persuasiveness, rebuttal, total, fallacies (JSON), tips (JSON)	One score row per user turn — the app's differentiator.

```
// prisma/schema.prisma (core models)
model User { id String @id @default(cuid()) email String @unique passwordHash String
             createdAt DateTime @default(now()) debates Debate[] }
model Debate { id String @id @default(cuid()) userId String user User @relation(fields:
[userId],references:[id])
              topic String userSide String aiSide String status String @default("active")
              createdAt DateTime @default(now()) turns Turn[] }
model Turn { id String @id @default(cuid()) debateId String debate Debate @relation(fields:
[debateId],references:[id])
            speaker String content String turnNumber Int createdAt DateTime @default(now()) score
Score? }
model Score { id String @id @default(cuid()) turnId String @unique turn Turn @relation(fields:
[turnId],references:[id])
             logic Int evidence Int persuasiveness Int rebuttal Int total Int
             fallacies Json tips Json createdAt DateTime @default(now()) }
```

6. The AI Logic (the heart of the app)

Role A — The Debater

System prompt locks the model to the assigned side, keeps rebuttals short (2–4 sentences), argues persuasively but fairly, and never concedes.

```
SYSTEM (Debater):
"You are debating the {aiSide} side of the topic: '{topic}'.
Argue persuasively for your side. Keep replies to 2-4 sentences.
Never concede or switch sides. Respond directly to the user's last point."
```

Role B — The Judge / Scorer

A second call that grades the user's argument and returns **strict JSON**. An explicit rubric (what a 1 vs a 10 means per category) keeps scores consistent.

```
SYSTEM (Judge):
"You are a debate coach. Score the user's argument on a 1-10 scale for:
logic, evidence, persuasiveness, rebuttal.
Detect logical fallacies (name + the exact quote + why).
Give 1-3 concrete improvement tips.
Respond with ONLY valid JSON in this exact shape:"

{
  "scores": { "logic": 7, "evidence": 5, "persuasiveness": 8, "rebuttal": 6 },
  "total": 26,
  "fallacies": [ { "name": "Straw Man", "quote": "...", "explanation": "..." } ],
  "tips": [ "Back your second claim with a concrete example." ]
}
```

JSON reliability: Because free HF models don't guarantee valid JSON, the backend wraps the Judge call in a helper that (1) extracts the first `{...}` block, (2) `JSON.parse`s it, and (3) if it fails, retries once with a "return valid JSON only" reminder before falling back to a safe default. This is in [services/scoring.ts](#).

7. Folder Structure & Which File Holds Which Logic

Backend — `/server`

Frontend — `/client`

```

server/
├─ src/
│   └─ index.ts          app entry, express
├─ setup
│   └─ routes/
│       ├── auth.ts      register / login
│       └─ debate.ts     start debate, take
├─ turn,
│   └─
├─ one
│   └─ services/
│       └─ llm.ts        callLLM() - HF
├─ provider
│   └─
│       ├── debater.ts   wrapper (swappable)
│                       builds Debater
├─ prompt
│   └─
│       ├── scoring.ts   Judge prompt + JSON
│                       parse/repair/retry
│       └─ middleware/
│           └─ auth.ts    JWT verify
├─ middleware
│   └─ prisma/
│       ├── schema.prisma DB models
│       └─ lib/prisma.ts  Prisma client
├─ singleton
├─ .env                  SECRETS (never
├─ committed)
└─ package.json

```

```

client/
├─ src/
│   ├── main.tsx         React entry
│   ├── App.tsx          routes
│   └─ pages/
│       ├── Home.tsx     topic + side picker
│       └─ Debate.tsx    live chat + score
├─ cards
│   ├── History.tsx      saved debates list
│   └─ components/
│       ├── ChatBox.tsx  message list + input
│       └─ ScoreCard.tsx score + fallacies +
├─ tips
│   └─ VoiceButton.tsx   Web Speech
├─ control
│   ├── hooks/
│   │   ├── useVoice.ts  speech-to-text / TTS
│   │   └─ useDebate.ts  API calls + state
│   └─ lib/api.ts        fetch wrapper (adds
├─ JWT)
│   └─ styles (Tailwind)
├─ .env                  VITE_API_URL only
└─ package.json

```

The swappable LLM layer (key design)

```

// server/src/services/llm.ts
export async function callLLM({ system, prompt, json }) {
  // Provider chosen by env var → HF today, Claude/local later, no app changes
  const res = await fetch(
    `https://api-inference.huggingface.co/models/${process.env.HF_MODEL}`,
    { method: "POST",
      headers: { Authorization: `Bearer ${process.env.HF_TOKEN}`,
        "Content-Type": "application/json" },
      body: JSON.stringify({ inputs: buildChat(system, prompt),
        parameters: { max_new_tokens: json ? 500 : 300,
          temperature: json ? 0.2 : 0.7,
          return_full_text: false } }) });

  const data = await res.json();
  return data[0].generated_text;
}

```


8. Credentials & Environment Variables Needed

Credential	Where to get it	Used in
HF_TOKEN	huggingface.co → Settings → Access Tokens → New token (Read role). Free.	Backend <code>.env</code> — auth for the HF Inference API.
HF_MODEL	The model id string, e.g. <code>meta-llama/Llama-3.3-70B-Instruct</code> .	Backend <code>.env</code> .
DATABASE_URL	neon.tech → create project → copy the Postgres connection string. Free tier.	Backend <code>.env</code> — Prisma connection.
JWT_SECRET	Any long random string you generate yourself.	Backend <code>.env</code> — sign/verify auth tokens.
PORT	e.g. <code>4000</code> (local).	Backend <code>.env</code> .
VITE_API_URL	Your backend URL (local: <code>http://localhost:4000</code> ; prod: the Render URL).	Frontend <code>.env</code> .

```
// server/.env (NEVER commit – add to .gitignore)
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXX
HF_MODEL=meta-llama/Llama-3.3-70B-Instruct
DATABASE_URL=postgresql://user:pass@ep-xxx.neon.tech/db?sslmode=require
JWT_SECRET=change_this_to_a_long_random_string
PORT=4000

// client/.env
VITE_API_URL=http://localhost:4000
```

Add `.env` to `.gitignore` in **both** folders. Commit a `.env.example` with empty values so reviewers know what's needed without seeing your secrets.

9. API Endpoints

Method + Path	Auth	Does
POST /api/auth/register	No	Create account (hash password with bcrypt), return JWT.
POST /api/auth/login	No	Verify credentials, return JWT.
POST /api/debate	Yes	Start a debate: save topic + sides, return debateId.
POST /api/debate/:id/turn	Yes	Core call: save user turn → Debater + Judge → save AI turn + score → return rebuttal + score.
GET /api/debate	Yes	List the user's saved debates.
GET /api/debate/:id	Yes	Full transcript + all scores of one debate.

10. Build Plan — Phases & Milestones (~2 weeks part-time)

Phase	Work	Milestone
0. Setup (½ day)	Two folders, Vite React app, Express server, Prisma + Neon, .env, health-check route.	Frontend ↔ backend ↔ DB all connected.
1. Core debate (2–3 d)	Topic/side picker, callLLM + Debater, chat UI, take-turn endpoint.	You can have a full text debate with the AI.
2. Scoring ★ (2–3 d)	Judge call, JSON parse/repair, Score table, ScoreCard UI with fallacies + tips.	The unique feature works — the headline.
3. Auth + history (1–2 d)	Register/login, JWT middleware, save + reopen debates, final summary screen.	Full CRUD + accounts.
4. Voice (1–2 d)	Web Speech API: speak arguments, AI reads rebuttals aloud.	The "wow" demo feature.
5. Polish + deploy (1–2 d)	Loading/error states, rate limiting, mobile responsive, deploy 3 tiers, README + demo GIF.	Live URL for the resume.

Minimum viable resume project = Phases 1–2. Phases 3–5 make it stand out.

11. Quick Setup Commands

```
# backend
mkdir ai-debate && cd ai-debate
mkdir server && cd server
npm init -y
npm i express cors dotenv jsonwebtoken bcrypt @prisma/client
npm i -D typescript ts-node-dev @types/express @types/node prisma
npx prisma init          # then paste schema + DATABASE_URL, run: npx prisma migrate dev

# frontend (from ai-debate root)
npm create vite@latest client -- --template react-ts
cd client && npm i && npm i -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

12. Watch Out For

- **API key security** — HF token backend-only, never in the React bundle.
- **HF rate limits / cold starts** — the first call to a serverless model can be slow ("model loading"). Show a loading state and retry after a short wait.
- **JSON reliability** — always run the Judge output through the parse/repair helper; never trust raw model text.
- **Cost control** — cap turns per debate and set `max_new_tokens` low.
- **Web Speech API support** — best in Chrome. Note this in the README instead of fighting cross-browser gaps.
- **Scoring consistency** — give the Judge an explicit 1-vs-10 rubric so scores are stable across turns.